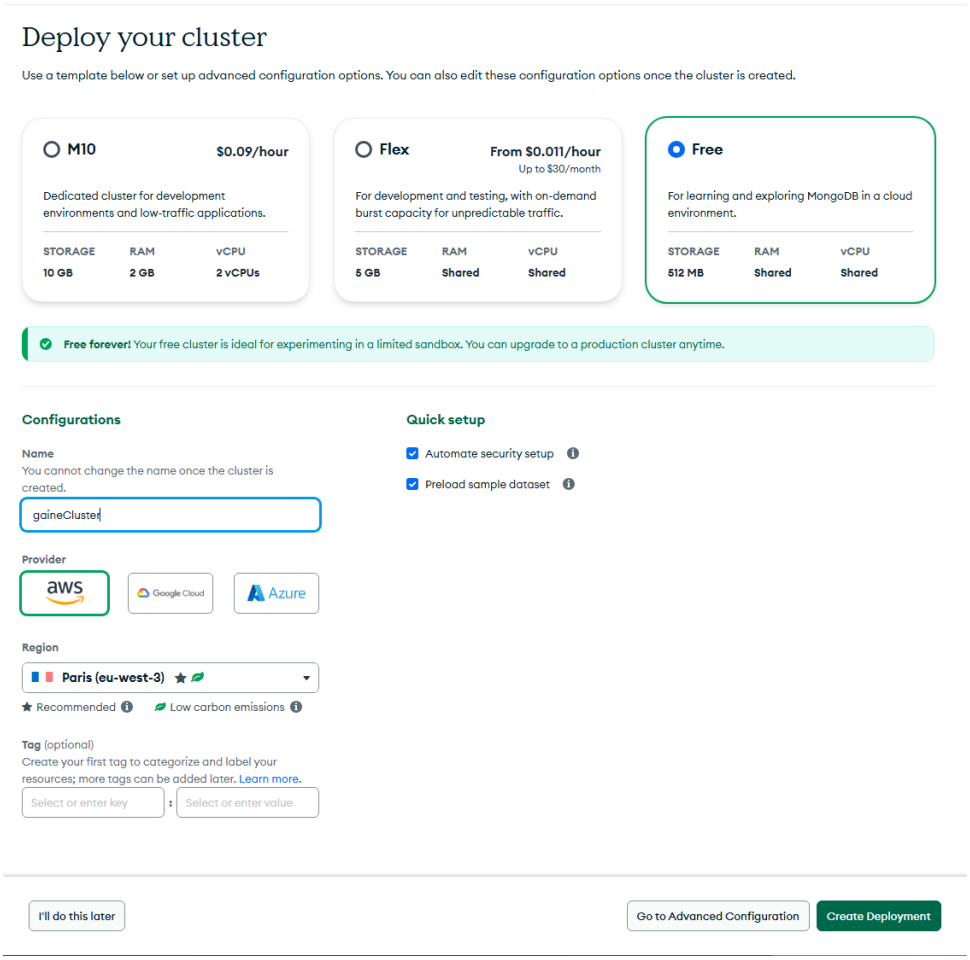
Práctica 3

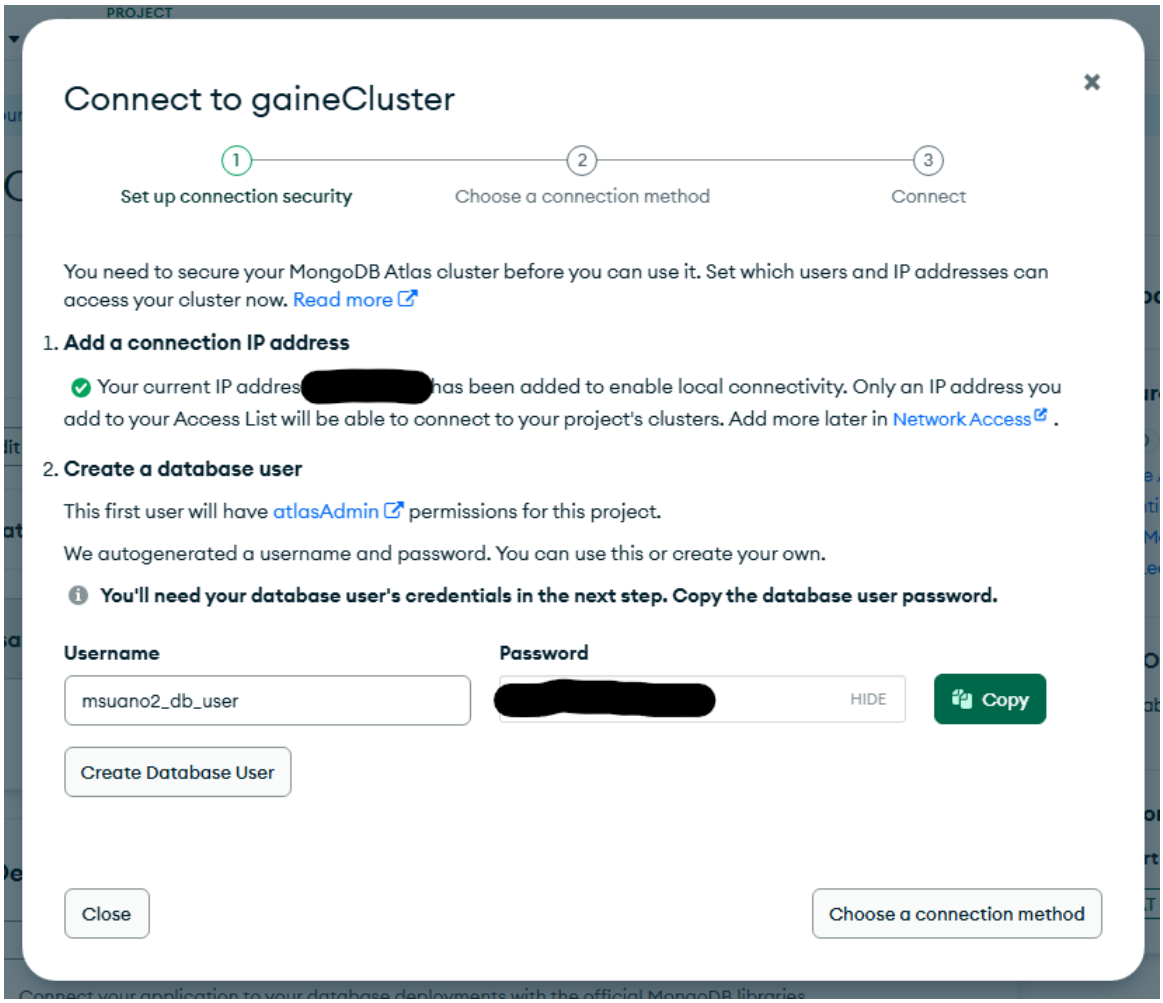
Manuel Suano Gallardo

Ejercicio 1

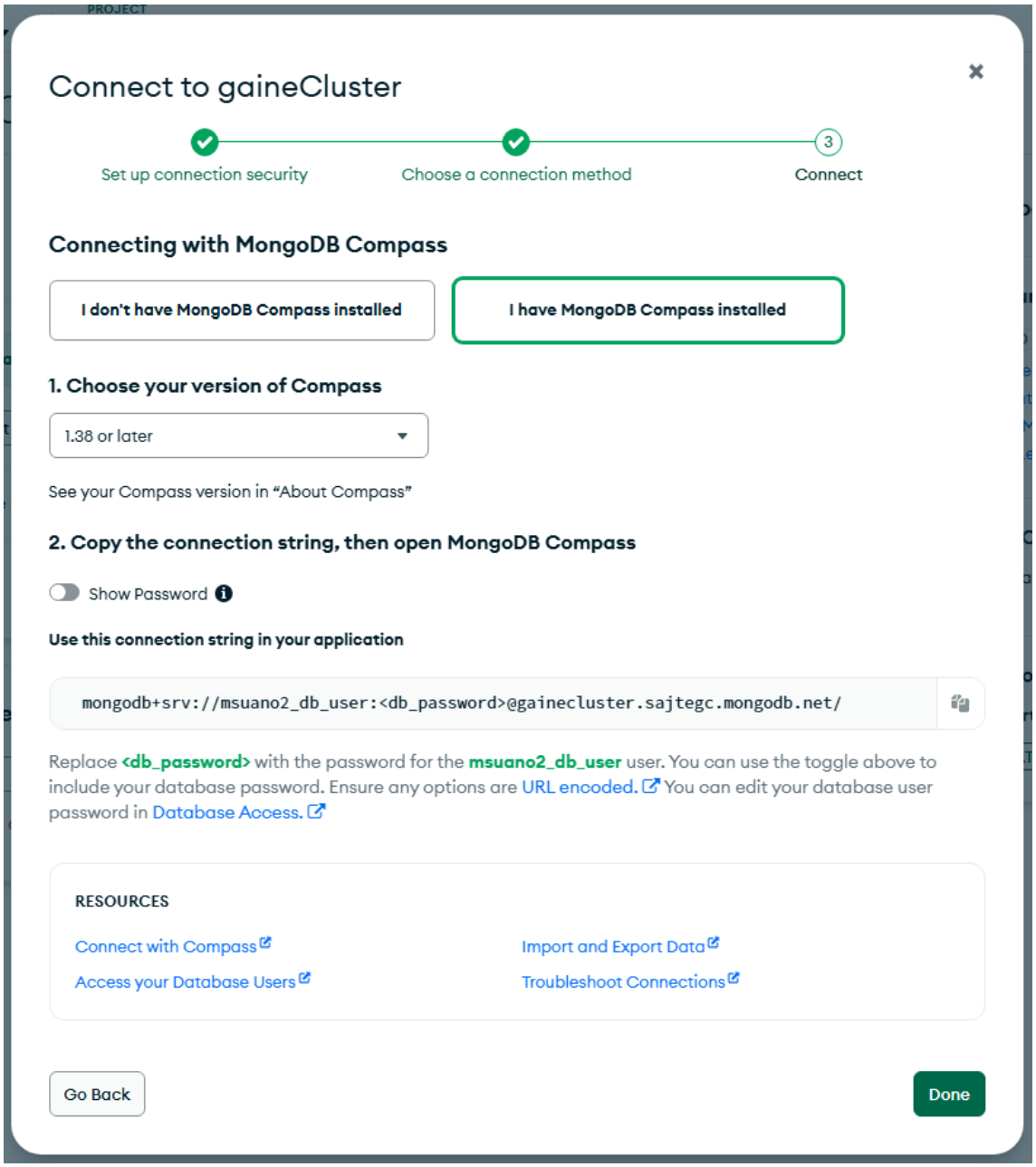
Tras crear la cuenta se pasa a la creación y despliegue del clúster. Seleccionando el plan gratis e indicando el nombre del clúster bastaría.



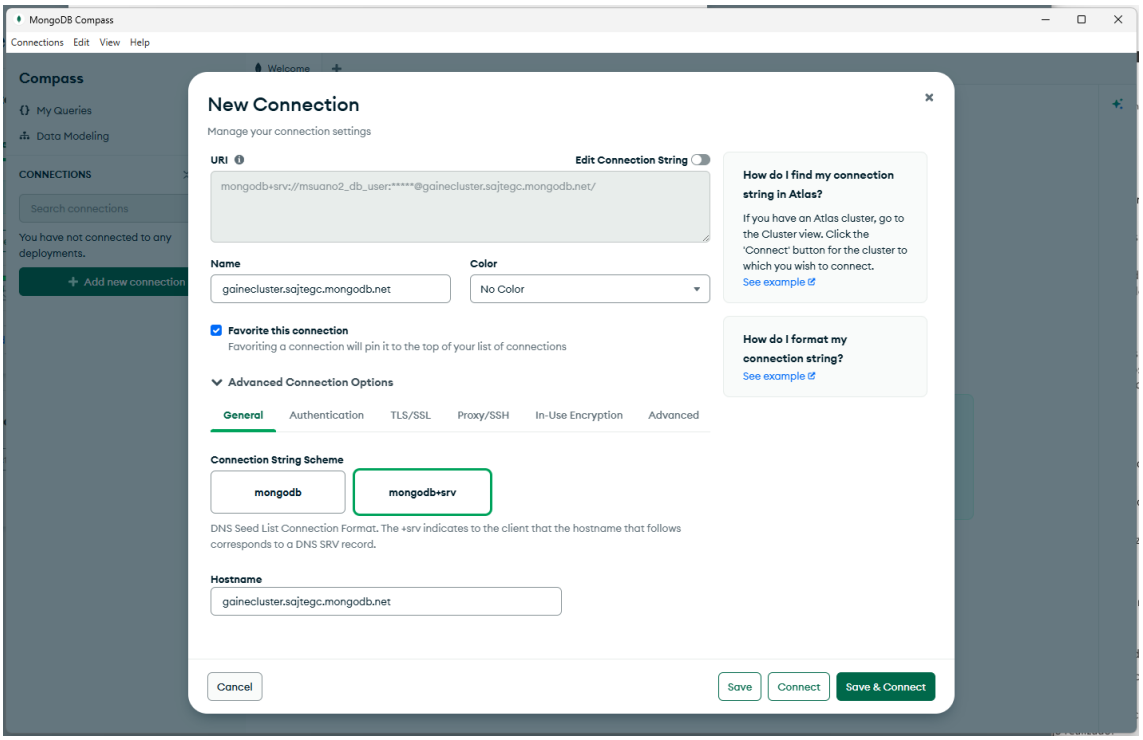
Tal y como se crea ofrece la opción de crear el usuario administrador:



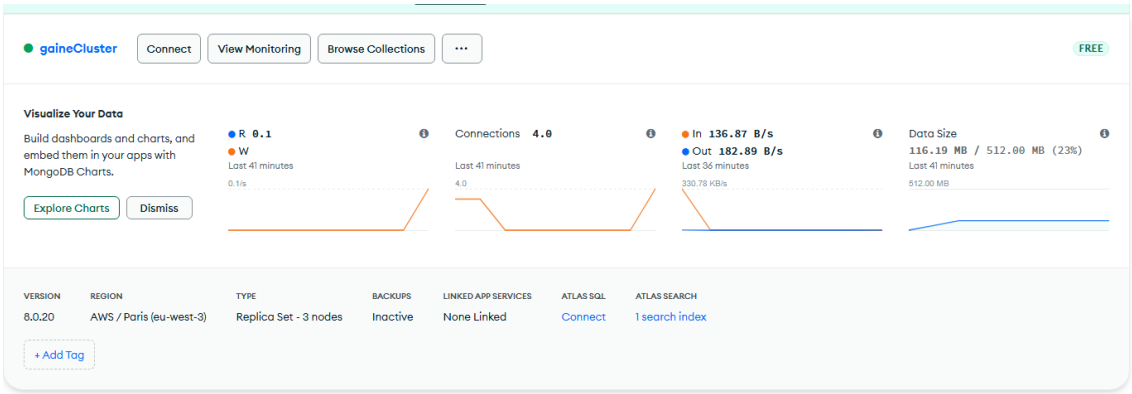
Al crear el usuario permitirá seleccionar la forma de conectarnos a la base de datos. Seleccionamos MongoDB Compass.



En la aplicación indicaremos dicha URL que se nos proporciona:

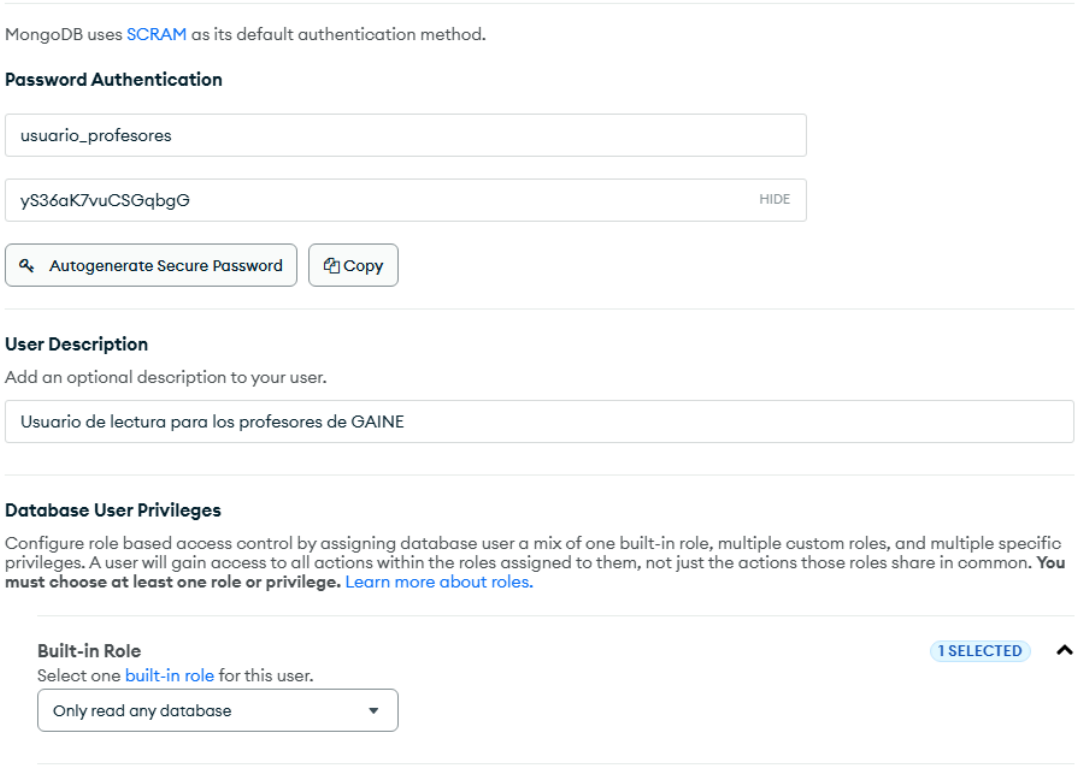


Una vez realizada la conexión, podemos ver en MongoDB Atlas la información del clúster: la región, el número de nodos, las conexiones y los índices entre otra información.



Pasamos a la creación del usuario de solo lectura para la base de datos. Accedemos al menú lateral, a la sección “Database & Network Access” y pulsamos el botón de crear nuevo usuario de la base de datos. Este usuario será:

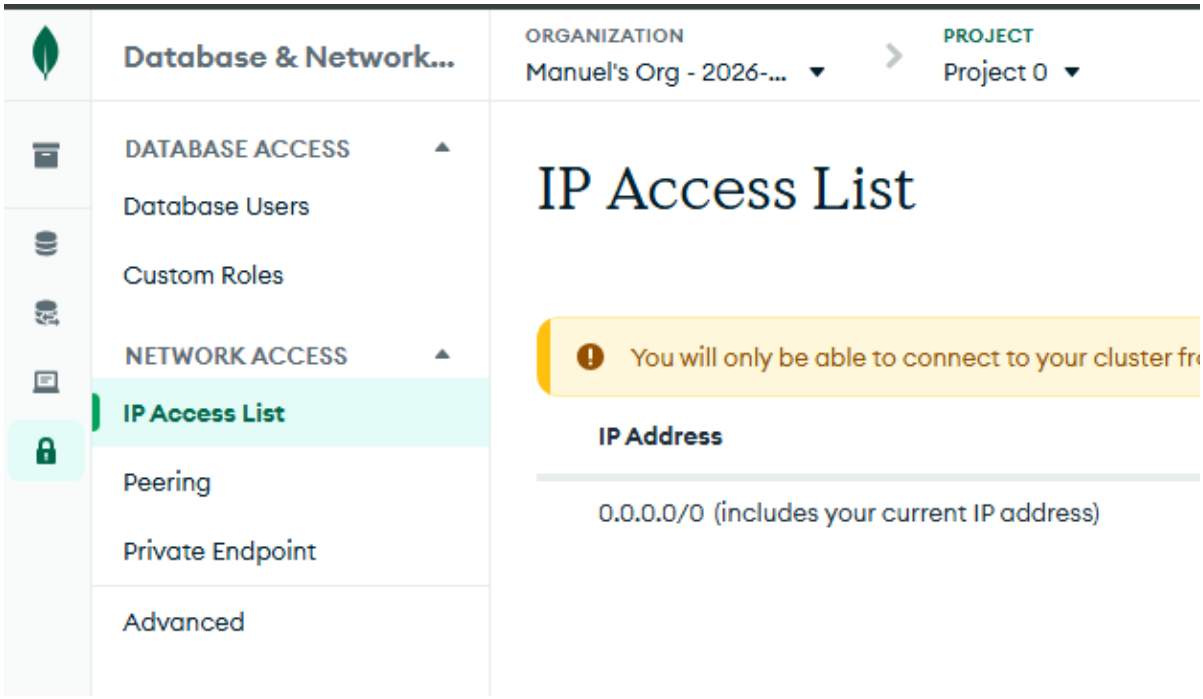
- usuario_profesores
- yS36aK7vuCSGqbgG



Para conectaros usando MongoDB Compass:

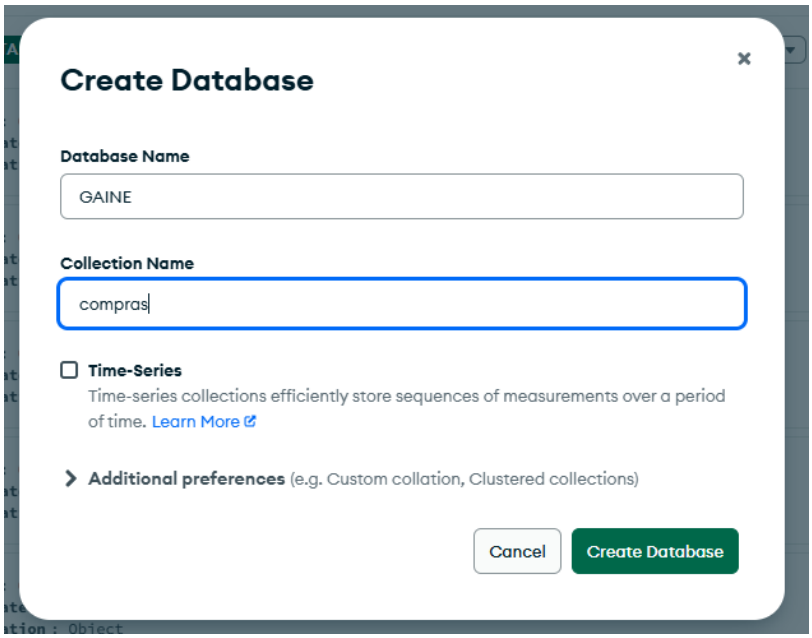
mongodb+srv://usuario_profesores:yS36aK7vuCSGqbgG@gainecluster.sajtegc.mongodb.net/

También es importante indicar que cualquier IP pueda conectarse a la base de datos. Para ello habrá que indicar la IP 0.0.0.0/0 en la lista de IPs que tienen acceso:



Ejercicio 2

Para importar el archivo “compras.json” en Mongo primero crearé una nueva base de datos, llamada “GAINE”, que contendrá la colección “compras” requerida en la práctica.



Con los datos importados se pueden realizar las consultas:

1. **Obtener todas las compras realizadas por el cliente “Clara” en las que el total gastado sea superior a 1000€, mostrando únicamente la fecha y el total de la compra.**

La consulta sería:

```
db.compras.find(
      {cliente: "Clara", total: {"$gt": 1000}},
      {fecha: 1, total: 1, _id: 0}
)
```

Se busca las instancias cuyo cliente sea “Clara” y el total de la compra sea mayor que 1000, usando el operador $gt. De esas instancias solo se ve la fecha y total, indicando cada campo con 1.

Ejemplo:

```
{
      fecha: '2024-08-22T00:00:00',
      total: 1216.37
}
```

2. **Obtener las compras que incluyan al menos un producto con nombre “SSD Externo”, mostrando el nombre del cliente, la fecha y el total.**

La consulta sería:

```
db.compras.find(
      {“productos.nombre”: "SSD Externo"}},
      {cliente: 1, fecha: 1, total: 1, _id: 0}
)
```

Se busca dentro del array productos si existe alguno que tenga por nombre “SSD externo”. De las instancias que se encuentren se selecciona el cliente, la fecha y el total.

Ejemplo:

```
{
      cliente: 'Elena',
      fecha: '2024-11-10T00:00:00',
      total: 792
}
```

3. **Encontrar las compras realizadas en el año 2024 cuyo total esté entre 500€ y 1500€, mostrando el cliente, la fecha y el total.**

La consulta sería:

```
db.compras.find(
      {
            fecha: {$regex: "2024"},
            total: {$gte: 500, $lte: 1500}
      },
      {cliente: 1, fecha: 1, total: 1, _id: 0}
)
```

Como se ha guardado la fecha en formato string, utilizo regex para buscar el año 2024. Por otro lado, busco las instancias que tengan el total mayor o igual que 500 y menor o igual que 1500.

Ejemplo:

```
{
      cliente: 'Elena',
      fecha: '2024-11-10T00:00:00',
      total: 792
}
```

4. **Obtener las compras que contengan algún producto “SSD Externo” cuya cantidad sea al menos 3 unidades y cuyo precio unitario sea inferior a 100€. Mostrar el cliente, la fecha y el total.**

La consulta sería:

```
db.compras.find(
     {productos: {
          $elemMatch: {
               nombre: "SSD Externo",
               cantidad: {$gte: 3},
               precio_unitario: {$lt: 100}
          }
     }},
     {cliente: 1, fecha: 1, total: 1, _id: 0}
)
```

En este caso se está buscando dentro de productos un elemento que cumpla las condiciones de que: su nombre sea SSD Externo, la cantidad sea mayor o igual que 3 y que el precio unitario sea menor estricto de 100.

Ejemplo:

```
{
     cliente: 'Javier',
     fecha: '2024-12-20T00:00:00',
     total: 244.25
}
```

5. **Obtener las compras en las que se hayan comprado a la vez los productos “Teclado mecánico” y “Ratón inalámbrico”. Mostrar el cliente, la fecha y el array de productos.**

La consulta sería:

```
db.compras.find(
    {$and: [
        {productos: {$elemMatch: {nombre: "Teclado
mecánico"}}},
        {productos: {$elemMatch: {nombre: "Ratón
inalámbrico"}}}
    ]},
    {cliente: 1, productos: 1, fecha: 1, _id: 0}
)
```

En este caso usaremos la conjunción para hacer cumplir la condición de que haya esos dos productos. A partir de las instancias que se obtengan se mostrará solo el cliente, la fecha y los productos comprados.

Ejemplo:

```
{
    cliente: 'Carlos',
    fecha: '2023-01-23T00:00:00',
    productos: [
        {
            nombre: 'Ratón inalámbrico',
            cantidad: 2,
            precio_unitario: 155.81
        },
        {
            nombre: 'Power Bank',
            cantidad: 3,
            precio_unitario: 182.29
        },
        {
            nombre: 'Teclado mecánico',
            cantidad: 5,
            precio_unitario: 195.98
        },
        {
            nombre: 'SSD Externo',
            cantidad: 2,
            precio_unitario: 152.98
        }
    ]
}
```

6. **Obtener las compras en las que el cliente haya comprado exactamente un único producto. Mostrar el cliente, la fecha, el total y el producto comprado.**

La consulta sería:

```
db.compras.find(
     {productos: {$size: 1}},
     {cliente: 1, fecha: 1, total: 1, productos: 1, _id:
0}
)
```

Buscamos aquellas instancias cuyo array asociado a productos sea de exactamente tamaño 1.

Ejemplo:

```
{
    cliente: 'Clara',
    fecha: '2024-03-19T00:00:00',
    total: 421.65,
    productos: [
        {
            nombre: 'Funda para móvil',
            cantidad: 3,
            precio_unitario: 140.55
        }
    ]
}
```

7. **Obtener las compras en las que todos los productos comprados tengan un precio unitario mayor o igual que 100€. Mostrar el cliente, la fecha y los productos.**

La consulta sería:

```
db.compras.find(
    {productos: {$not: {$elemMatch: {precio_unitario: {$lt: 100}}}}},
    {cliente: 1, fecha: 1, productos: 1, _id: 0}
)
```

En este caso se usa en la condición la búsqueda de un producto con un precio unitario menor que 100, solo que negada, ya que de esta forma nos aseguramos de que todos sean mayor o igual.

Ejemplo:

```
{
    cliente: 'Elena',
    fecha: '2024-11-10T00:00:00',
    productos: [
        {
            nombre: 'Ratón inalámbrico',
            cantidad: 3,
            precio_unitario: 108.28
        },
```

```
        {
            nombre: 'SSD Externo',
            cantidad: 4,
            precio_unitario: 116.79
        }
    ]
}
```

Y, a continuación, las agregaciones:

1. **Obtener todas las compras realizadas por el cliente “Luis” utilizando el framework de agregación.**

La agregación sería:

```
db.compras.aggregate([
    {$match: {
        "cliente”: "Luis"
    }}
])
```

Se seleccionan aquellas instancias que tengan por valor en el campo “cliente” a Luis. Esta agregación retorna vacío, pues no hay compras cuyo cliente sea Luis.

2. **Obtener las compras cuyo total sea mayor que 1000€, mostrando solo el cliente y el total.**

La agregación sería:

```
db.compras.aggregate([
    {$match: {
        "total”: {
            $gt: 1000
        }
    }},
    {$project: {
        "cliente”: 1,
        "total”: 1,
        "_id”: 0
    }}
])
```

Se buscan aquellas instancias que cumplan que el campo total tiene un valor mayor estricto a 1000 y se seleccionan solo el cliente y el total de la compra.

Ejemplo:

```
{
      cliente: 'Fernando',
      total: 1185.29
}
```

3. **Calcular el importe total gastado por cada cliente, mostrando el nombre del cliente y el total acumulado.**

La agregación sería:

```
db.compras.aggregate([
      {$group: {
            _id: "$cliente",
            total: {
                  $sum: "$total"
            }
      }}
])
```

Se agrupa por nombre de cliente y se suman los totales de cada una de las compras bajo la clave total.

Ejemplo:

```
{
      _id: 'Clara',
      total: 9865.15
}
```

4. **Obtener la cantidad total de unidades vendidas de cada producto, ordenando el resultado de mayor a menor número de unidades vendidas.**

La agregación sería:

```
db.compras.aggregate([
      {$unwind: {
            path: "$productos"
      }},
      {$group: {
            _id: "$productos.nombre",
            cantidad_total: {
                  $sum: "$productos.cantidad"
            }
      }},
      {$sort: {
            cantidad_total: -1
```

```
        }}
    ])
```

En este caso se usa la instrucción $unwind sobre el array de productos, de forma que para cada producto del array se genera una nueva instancia. A partir de esa separación se puede agrupar por nombre de producto y sumar la cantidad de productos vendidos. Finalmente se ordena de forma descendente según dicha cantidad.

Ejemplo, los tres productos más vendidos:

```
{
        _id: 'SSD Externo',
        cantidad_total: 52
}
{
        _id: 'Altavoces Bluetooth',
        cantidad_total: 48
}
{
        _id: 'Teclado mecánico',
        cantidad_total: 41
}
```

5. **Obtener, para cada compra, el número total de unidades compradas en esa compra, mostrando el cliente, la fecha, el total y el total de unidades. Ordenar el resultado de mayor a menor número de unidades.**

La agregación sería:

```
db.compras.aggregate([
        {$project: {
                cliente: 1,
                fecha: 1,
                total: 1,
                total_unidades: {$sum: "$productos.cantidad"},
                _id: 0,
        }}
])
```

Se muestran todos los campos requeridos y se añade el nuevo total de unidades compradas. Esto se hace con la instrucción $sum, que toma todos los valores del array productos y los suma.

Ejemplo:

```
{
```

```
    cliente: 'Elena',
    fecha: '2024-11-10T00:00:00',
    total: 792,
    total_unidades: 7
}
```

6. **Obtener el importe total gastado por cada cliente en compras superiores a 1000€, mostrando el nombre del cliente y el total gastado en ese tipo de compras.**

La agregación quedaría

```
db.compras.aggregate([
    {$match: {
        "total": {
            $gt: 1000
        }
    }},
    {$group: {
        _id: "$cliente",
        total_compras: {
            $sum: "$total"
        }
    }},
    {$project: {
        _id: 0,
        cliente: "$_id",
        total_gastado: 1
    }}
])
```

Seleccionamos solo aquellas compras que sean superiores a 1000 euros con el operador $match y después agruparemos por cliente para obtener la suma total de esas compras. Por último se mostrará la información cambiando el campo "_id" por cliente, para que sea más intuitivo visualmente.

Ejemplo:

```
{
    total_compras: 3836.66,
    cliente: 'Fernando'
}
```

7. **Obtener el producto que más dinero ha generado en ventas, mostrando su nombre y el importe total facturado por ese producto.**

La agregación sería:

```
db.compras.aggregate([
	{$unwind: "$productos"},
	{$group: {
		_id: "$productos.nombre",
		total: {
			$sum: {$multiply: ["$productos.cantidad",
"$productos.precio_unitario"]}
		}
	}},
	{$sort: {total: -1}},
	{$limit: 1},
	{$project: {
		producto: "$_id",
		total: 1,
		_id: 0
	}}
])
```

En este caso volvemos a generar una instancia por cada elemento del vector. A partir de todas las instancias agrupamos por el nombre del producto, añadiendo el valor generado en esa compra multiplicando el precio unitario por la cantidad de elementos vendidos. Ordenamos descendentemente por precio y nos quedamos con el primero, que será el que más ha generado en ventas.

El producto sería el teclado mecánico:

```
{
	total: 5642.23,
	producto: 'Teclado mecánico'
}
```

Ejercicio 3

La base de datos contendrá 4 colecciones:

- Una para guardar la información de los usuarios. Las instancias de esta colección deberán tener los siguientes campos, además del _id automático:
  - **`nombre`**. Nombre de usuario.
  - **`correo`**. Correo electrónico del usuario.
  - **`fecha_registro`**. Fecha en la que se registró el usuario en la plataforma. Será un timestamp UNIX. Formato más amplio para adaptarlo según la zona horaria.

- **favoritos**. Array que contendrá los IDs de las recetas que el usuario indique como favoritas.

Por ejemplo:

```
{
    nombre: “manusuga”,
    correo: “msuano2@alumno.uned.es”,
    fecha_registro: 1767222000,
    favoritos: [ .....]
}
```

- Otra colección para guardar las recetas publicadas. En este caso se guardará la siguiente información de cada una de las recetas. Los campos que habría serían:
  - **titulo**. Título de la receta.
  - **descripcion**. Texto informativo acerca de la receta.
  - **autor**. ID del autor que ha publicado la receta.
  - **fecha**. Timestamp UNIX de la fecha en la que se creó la receta.
  - **ingredientes**. Lista de ingredientes que contiene la receta. Para cada ingrediente se almacenan la siguiente información:
    - **ingrediente**. Nombre del ingrediente usado.
    - **cantidad**. Cantidad usada en la receta, solo el número.
    - **unidad**. Nomenclatura o texto indicando la unidad de la cantidad.
  - **pasos**. Lista de pasos a seguir para realizar la receta. Para cada paso de almacena:
    - **orden**. Orden del paso a la hora de realizar la receta.
    - **descripcion**. En qué consiste el paso.
  - **dificultad**. Grado de dificultad de la receta.
  - **tiempo**. Tiempo en minutos que se tarda en realizar la receta.
  - **etiquetas**. Lista de etiquetas para dar contexto a la receta.

Por ejemplo:

```
{
    titulo: “Huevos duros”,
    descripcion: “Huevos cocidos en agua”,
    autor: .....,
    fecha: 1767223000,
    ingredientes: [
        {
            ingrediente: “huevo”,
            cantidad: 1,
            unidad: “unidad”
        }
    ],
```

```
            pasos: [
                  {
                        orden: 1,
                        descripcion: “Meter el huevo en una
cacerola pequeña y llenar de agua hasta que cubra el huevo”
                  },
                  {
                        orden: 2,
                        descripcion: “Calentar agua hasta que
comience a hervir”
                  },
                  {
                        orden: 3,
                        descripcion:  “Cuando  comience  a
hervir, esperar 10 minutos aproximadamente”
                  },
                  {
                        orden: 4,
                        descripcion:  “Vaciar  la  cacerola  y
echar agua fría”
                  },
                  {
                        orden: 5,
                        descripcion: “Pelar el huevo”
                  }
            ],
            dificultad: “muy fácil”,
            tiempo: “15”,
            etiquetas: [“huevo”, “cocer”]
      }
```

- Para las valoraciones de las recetas por parte de los usuarios habrá otra colección. Estas valoraciones son una relación de muchos a muchos en la que un usuario puede valorar desde muchas rectas a ninguna, y una receta puede ser tener desde muchas valoraciones hasta ninguna, por lo que lo recomendado sería crear una colección para guardar estar interacciones. En esta colección, cada instancia tendrá la siguiente información:
    - **receta**. ID de la receta.
    - **usuario**. ID del usuario.
    - **valoracion**. Valor de puntuaje decidido para a realización de las valoraciones. En nuestro caso serán estrellas y podrán ser como máximo 5.

- **fecha**. Timestamp UNIX de la fecha en la que se ha realizado la valoración de la receta.

Por ejemplo:

```
{
    receta: ObjectId(...),
    usuario: ObjectId(...),
    valoracion: 5,
    fecha: 1767224000
}
```

- De igual forma ocurre con los comentarios, por lo que habrá otra colección para almacenarlos. La información que se almacenará será:
  - **receta**. ID de la receta.
  - **usuario**. ID de la receta.
  - **texto**. Comentario realizado.
  - **fecha**. Timestamp UNIX de la fecha en la que se ha realizado el comentario.

Por ejemplo:

```
{
    receta: ObjectId(...),
    usuario: ObjectId(...),
    texto: “Muy fácil de seguir. ¡Ha quedado riquísimo!”,
    fecha: 1767225000
}
```

La utilización de referencias a ObjectID en lugar del nombre de usuario o del título de la receta en diferentes campos de las colecciones permite modificar unos de estos dos valores sin la necesidad de tener que recorrer toda la base de datos al completo buscando los valores antiguos y teniendo que actualizarlos.

Por otro lado, el uso de la fecha en timestamp de UNIX permite la ordenación de las instancias usando números en lugar de estar usando cadenas de caracteres, así como tener un valor universal sin tener en cuenta el huso horario.

En cada una de las colecciones existirán 5 elementos, insertados con las siguientes instrucciones:

- **Para la colección de usuarios**:

```
db.usuarios.insertMany([
    {
        _id: ObjectId(1),
        nombre: "manusuga",
        correo: "msuano2@alumno.uned.es",
        fecha_registro: 1767222000,
```

```
            favoritos: []
      },
      {
            nombre: "laurasuga",
            correo: "laurasuga@lookout.es",
            fecha_registro: 1767222000,
            favoritos: []
      },
      {
            nombre: "sofiasava",
            correo: "sofiasava@glegoo.com",
            fecha_registro: 1767222000,
            favoritos: []
      },{
            nombre: "daniel99",
            correo: "dan99el@hooya.es",
            fecha_registro: 1767222000,
            favoritos: []
      },
      {
            nombre: "luci72gasa",
            correo: "luciagasa@glegoo.com",
            fecha_registro: 1767222000,
            favoritos: []
      }
])
```

- **Para la colección de recetas**:

```
db.recetas.insertMany([
      {
            titulo: "Huevos duros",
            descripcion: "Huevos cocidos en agua",
            autor: ObjectId('69d28fffae6a3bb9bbf987dd'),
            fecha: 1767240000,
            ingredientes: [
                  {
                        ingrediente: "huevo",
                        cantidad: 1,
                        unidad: "unidad"
                  }
            ],
            pasos: [
                  {
                        orden: 1,
```

```
                    descripcion: "Meter el huevo en una
cacerola pequeña y llenar de agua hasta que cubra el huevo"
                },
                {
                    orden: 2,
                    descripcion: "Calentar agua hasta que
comience a hervir"
                },
                {
                    orden: 3,
                    descripcion:   "Cuando   comience   a
hervir, esperar 10 minutos aproximadamente"
                },
                {
                    orden: 4,
                    descripcion:  "Vaciar  la  cacerola  y
echar agua fría"
                },
                {
                    orden: 5,
                    descripcion: "Pelar el huevo"
                }
            ],
            dificultad: "muy fácil",
            tiempo: 15,
            etiquetas: ["huevo", "cocer"]
        },
        {
            titulo: "Pan con tomate",
            descripcion: "La clásica y deliciosa tostada de
desayuno español",
            autor: ObjectId('69d28fffae6a3bb9bbf987dd'),
            fecha: 1767245000,
            ingredientes: [
                {
                    ingrediente: "pan",
                    cantidad: 1,
                    unidad: "rebanada"
                },
                {
                    ingrediente: "tomate maduro",
                    cantidad: 0.5,
                    unidad: "unidad"
```

```
            },
            {
                ingrediente: "aceite de oliva virgen extra",
                cantidad: 10,
                unidad: "ml"
            },
            {
                ingrediente: "sal",
                cantidad: 1,
                unidad: "pizca"
            }
        ],
        pasos: [
            {
                orden: 1,
                descripcion: "Tostar la rebanada de pan al
gusto"
            },
            {
                orden: 2,
                descripcion: "Rallar el tomate y extenderlo
sobre el pan tostado"
            },
            {
                orden: 3,
                descripcion: "Añadir un chorrito de aceite
de oliva y la pizca de sal por encima"
            }
        ],
        dificultad: "muy fácil",
        tiempo: 5,
        etiquetas: ["desayuno", "tostada", "mediterráneo"]
    },
    {
        titulo: "Quesadilla exprés",
        descripcion: "Tortilla de trigo rellena de queso
fundido",
        autor: ObjectId('69d28fffae6a3bb9bbf987dd'),
        fecha: 1767250000,
        ingredientes: [
            {
                ingrediente: "tortilla de trigo",
                cantidad: 1,
```

```
                unidad: "unidad"
            },
            {
                ingrediente: "queso rallado para fundir",
                cantidad: 50,
                unidad: "gramos"
            }
        ],
        pasos: [
            {
                orden: 1,
                descripcion: "Colocar la tortilla en una
sartén caliente a fuego medio"
            },
            {
                orden: 2,
                descripcion: "Poner el queso en una mitad de
la tortilla y doblarla por la mitad"
            },
            {
                orden: 3,
                descripcion: "Dorar por ambos lados hasta
que el queso esté completamente fundido"
            }
        ],
        dificultad: "muy fácil",
        tiempo: 4,
        etiquetas: ["cena rápida", "queso", "snack"]
    },
    {
        titulo: "Yogur con miel y nueces",
        descripcion: "Un postre o merienda saludable y súper
rápido",
        autor: ObjectId('69d28fffae6a3bb9bbf987de'),
        fecha: 1767240000,
        ingredientes: [
            {
                ingrediente: "yogur natural o griego",
                cantidad: 1,
                unidad: "unidad"
            },
            {
                ingrediente: "miel",
```

```
                cantidad: 1,
                unidad: "cucharada"
            },
            {
                ingrediente: "nueces peladas",
                cantidad: 15,
                unidad: "gramos"
            }
        ],
        pasos: [
            {
                orden: 1,
                descripcion: "Verter el yogur en un bol
pequeño"
            },
            {
                orden: 2,
                descripcion: "Añadir las nueces picadas con
las manos y coronar con el hilo de miel"
            }
        ],
        dificultad: "muy fácil",
        tiempo: 2,
        etiquetas: ["postre", "saludable", "sin cocinar"]
    },
    {
        titulo: "Batido de plátano clásico",
        descripcion: "Bebida cremosa y energética para
cualquier momento",
        autor: ObjectId('69d28fffae6a3bb9bbf987de'),
        fecha: 1767245000,
        ingredientes: [
            {
                ingrediente: "plátano maduro",
                cantidad: 1,
                unidad: "unidad"
            },
            {
                ingrediente: "leche",
                cantidad: 200,
                unidad: "ml"
            }
        ],
```

```
        pasos: [
            {
                orden: 1,
                descripcion: "Pelar el plátano y trocearlo
dentro del vaso de la batidora"
            },
            {
                orden: 2,
                descripcion: "Añadir la leche y triturar
todo hasta que quede una mezcla homogénea"
            }
        ],
        dificultad: "muy fácil",
        tiempo: 3,
        etiquetas: ["bebida", "fruta", "merienda"]
    }
])
```

Estas recetas las he generado usando Gemini con el objetivo de reducir el tiempo invertido.

- **Para la colección de valoraciones**:

```
db.valoraciones.insertMany([
     {
          receta: ObjectId('69d2a3feae6a3bb9bbf987e2'),
          usuario: ObjectId('69d28fffae6a3bb9bbf987df'),
          valoracion: 5,
          fecha: 1767260000
     },
     {
          receta: ObjectId('69d2a3feae6a3bb9bbf987e3'),
          usuario: ObjectId('69d28fffae6a3bb9bbf987df'),
          valoracion: 2,
          fecha: 1767261000
     },
     {
          receta: ObjectId('69d2a3feae6a3bb9bbf987e4'),
          usuario: ObjectId('69d28fffae6a3bb9bbf987df'),
          valoracion: 0,
          fecha: 1767262000
     },
     {
          receta: ObjectId('69d2a3feae6a3bb9bbf987e2'),
          usuario: ObjectId('69d28fffae6a3bb9bbf987e0'),
          valoracion: 5,
```

```
            fecha: 1767260000
      },
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e6'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987e0'),
            valoracion: 5,
            fecha: 1767260000
      }
])
```

- **Para la colección de comentarios**:

```
db.comentarios.insertMany([
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e5'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987e1'),
            texto: "Muy buena receta, a mi hijo le ha encantado",
            fecha: 1767260000
      },
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e4'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987e1'),
            texto: "Genial, perfecta para cuando no tienes muchas ganas de cocinar",
            fecha: 1767261000
      },
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e3'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987df'),
            texto: "Plato clásico, se le podría añadir cualquier ingrediente para conseguir mejores sabores",
            fecha: 1767262000
      },
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e2'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987e0'),
            texto: "Sencillo y muy beneficioso, gran aporte de proteínas",
            fecha: 1767260000
      },
      {
            receta: ObjectId('69d2a3feae6a3bb9bbf987e5'),
            usuario: ObjectId('69d28fffae6a3bb9bbf987e0'),
            texto: "Perfecto. Recomiendo usar yogur griego",
```

```
            fecha: 1767260000
        }
    ])
```

Toda esta información da las siguientes relaciones dentro de la BD:

- Existen 5 usuarios:
    - **manusuga**. ID 69d28fffae6a3bb9bbf987dd.
        - Publica las recetas:
            - **Huevos duros**. ID 69d2a3feae6a3bb9bbf987e2
            - **Pan con tomate**. ID 69d2a3feae6a3bb9bbf987e3.
            - **Quesadilla exprés**. ID 69d2a3feae6a3bb9bbf987e4.
        - Tiene como favorito la receta:
            - **Yogur con miel y nueces.**
    - **laurasuga**. ID 69d28fffae6a3bb9bbf987de.
        - Publica las recetas:
            - **Yogur con miel y nueces**. ID 69d2a3feae6a3bb9bbf987e5.
            - **Batido de plátano clásico**. ID 69d2a3feae6a3bb9bbf987e6.
    - **sofiasava**. ID 69d28fffae6a3bb9bbf987df.
        - Valora las recetas:
            - **Huevos duros** con 5 estrellas.
            - **Pan con tomate** con 2 estrellas.
            - **Quesadilla exprés** con 0 estrellas.
        - Comenta en las recetas:
            - **Pan con tomate** el texto: “Plato clásico, se le podría añadir cualquier ingrediente para conseguir mejores sabores”.
        - Tiene por favoritos la receta:
            - **Huevos duros.**
    - **daniel99**. ID 69d28fffae6a3bb9bbf987e0.
        - Valora las recetas:
            - **Huevos duros** con 5 estrellas.
            - **Batido de plátano clásico** con 5 estrellas.
        - Comenta en las recetas:
            - **Huevos duros** el texto: “Sencillo y muy beneficioso, gran aporte de proteínas”.
            - **Batido de plátano clásico** el texto: “Perfecto. Recomiendo usar yogur griego”.
        - Tiene por favoritos la receta:

            - **Huevos duros.**
            - **Batido de plátano clásico.**
    - **luci72gasa**. ID 69d28fffae6a3bb9bbf987e1.
        - Comenta en las recetas:
            - **Yogur con miel y nueces** el texto: “Muy buena receta, a mi hijo le ha encantado”.
            - **Quesadilla exprés** el texto: “Genial, perfecta para cuando no tienes muchas ganas de cocinar”.

Para la adición de los favoritos a cada usuario se usa las siguientes instrucciones:

- Para el usuario manusuga:

```
db.usuarios.updateOne({_id:
ObjectId("69d28fffae6a3bb9bbf987dd")}, {$set:
{favoritos: [ObjectId("69d2a3feae6a3bb9bbf987e5")]}})
```

- Para el usuario sofiasava:

```
db.usuarios.updateOne({_id:
ObjectId("69d28fffae6a3bb9bbf987df")}, {$set:
{favoritos: [ObjectId("69d2a3feae6a3bb9bbf987e2")]}})
```

- Para el usuario daniel99:

```
db.usuarios.updateOne({_id:
ObjectId("69d28fffae6a3bb9bbf987e0")}, {$set:
{favoritos: [ObjectId("69d2a3feae6a3bb9bbf987e2"),
ObjectId("69d2a3feae6a3bb9bbf987e6")]}})
```

La creación de los índices para las consultas más frecuentes se realizará mediante los comandos:

1. **Consultas por nombre o correo electrónico de usuario.**

Para este caso crearé un índice con los dos campos:

```
db.usuarios.createIndex({correo: 1, nombre: 1})
```

De esta forma se creará un índice con el correo electrónico y el nombre. Por ejemplo, si realizamos la consulta para buscar al usuario por nombre “manusuga”:

```
db.usuarios.find({nombre: "manusuga"})
```

Obtenemos el usuario al completo:

```
{
  _id: ObjectId('69d28fffae6a3bb9bbf987dd'),
  nombre: 'manusuga',
  correo: 'msuano2@alumno.uned.es',
  fecha_registro: 1767222000,
  favoritos: []
```

```
}
```

2. **Búsqueda de recetas que contengan un ingrediente específico.**

En este caso crearemos un índice a partir de la clave `ingrediente` que exista dentro del array `ingredientes`:

```
db.recetas.createIndex({"ingredientes.ingrediente": 1})
```

Al realizar entonces la consulta:

```
db.recetas.find({"ingredientes.ingrediente": "huevo"},
{titulo: 1; ingredientes: 1})
```

nos devolverá la receta de huevos duros:

```
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e2'),
  titulo: 'Huevos duros',
  ingredientes: [
    {
      ingrediente: 'huevo',
      cantidad: 1,
      unidad: 'unidad'
    }
  ]
}
```

3. **Búsqueda de recetas por nivel de dificultad o por tiempo máximo de preparación.**

Al igual que en el primer caso, crearemos un índice indicando la dificultad y el tiempo:

```
db.recetas.createIndex({dificultad: 1, tiempo: 1})
```

De esta forma, podremos hacer la consulta:

```
db.recetas.find({dificultad: “muy fácil”, tiempo: {$lte:
4}}, {titulo: 1, tiempo: 1})
```

Obtendríamos las tres recetas que tienen un tiempo inferior o igual a 4 minutos dificultad “muy fácil”, ordenadas por tiempo:

```
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e5'),
  titulo: 'Yogur con miel y nueces',
  tiempo: 2
}
{
```

```
  _id: ObjectId('69d2a3feae6a3bb9bbf987e6'),
  titulo: 'Batido de plátano clásico',
  tiempo: 3
}
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e4'),
  titulo: 'Quesadilla exprés',
  tiempo: 4
}
```

4. **Listado de recetas ordenadas por fecha de creación (las "n" últimas recetas publicadas, de la más reciente a la más antigua).**

En este caso el índice habría que crearlo con -1, pues se quiere de forma descendente:

```
db.recetas.createIndex({fecha: -1})
```

De esta forma, al realizar la consulta

```
db.recetas.find({}, {titulo: 1, fecha: 1}).sort({fecha: -1}).limit(2)
```

se escogen del índice las tres recetas más recientes:

```
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e4'),
  titulo: 'Quesadilla exprés',
  fecha: 1767250000
}
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e3'),
  titulo: 'Pan con tomate',
  fecha: 1767245000
}
```

5. **Obtención de las recetas mejor valoradas, calculando la media de las puntuaciones.**

Para este caso crearemos un índice que contenga dos claves: el ID de la receta y la puntuación dada.

```
db.valoraciones.createIndex({receta: 1, valoracion: 1})
```

De esta forma se almacenarán ambos valores en el índice y podremos realizar el cálculo de manera más rápida. Al realizar la agregación (pues es necesario agrupar por receta):

```
db.valoraciones.aggregate([
      {$match: {receta:
ObjectId(“69d2a3feae6a3bb9bbf987e2”)}},
      {$group: {_id: “$receta”, media: {$avg:
“$valoracion”}}}
])
```

Nos da que de media la receta de “Huevos duros” tiene una valoración de 5, pues dos usuarios la han valorado como un 5:

```
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e2'),
  media: 5
}
```

6. **Búsqueda de recetas por etiquetas.**

En este caso tendremos que crear un índice para el campo etiquetas:

```
db.recetas.createIndex({etiquetas: 1})
```

Podremos ejecutar consultas tal que:

```
db.recetas.find({etiquetas: {$in: [“postre”, “fruta”]}},
{titulo: 1, etiquetas: 1})
```

y nos mostraría las dos recetas que contienen algunas de esas dos etiquetas:

```
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e6'),
  titulo: 'Batido de plátano clásico',
  etiquetas: [
    'bebida',
    'fruta',
    'merienda'
  ]
}
{
  _id: ObjectId('69d2a3feae6a3bb9bbf987e6'),
  titulo: 'Batido de plátano clásico',
  etiquetas: [
    'bebida',
    'fruta',
    'merienda'
  ]
}
```

Extrayéndolas desde el índice.

7. **Contar cuántas veces una receta ha sido guardada como favorita.**

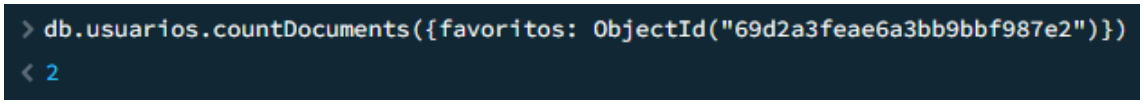
Habría que crear un índice de la clave “favoritos” de los usuarios:

```
db.usuarios.createIndex({favoritos: 1})
```

En este caso la consulta sería:

```
db.usuarios.countDocuments({favoritos:
ObjectId(“69d2a3feae6a3bb9bbf987e2”)})
```

que nos devuelve 2, pues hay dos usuarios que tiene como favorito la receta de los huevos duros.



8. **Obtener las valoraciones de una receta específica.**

En este caso sirve el mismo índice usado en el apartado 5. La consulta

```
db.valoraciones.find({receta:
ObjectId(“69d2a3feae6a3bb9bbf987e2”)})
```

nos proporcionaría las dos valoraciones que tiene la receta de los huevos duros.

```
{
  _id: ObjectId('69d2b044ae6a3bb9bbf987e7'),
  receta: ObjectId('69d2a3feae6a3bb9bbf987e2'),
  usuario: ObjectId('69d28fffae6a3bb9bbf987df'),
  valoracion: 5,
  fecha: 1767260000
}
{
  _id: ObjectId('69d2b044ae6a3bb9bbf987ea'),
  receta: ObjectId('69d2a3feae6a3bb9bbf987e2'),
  usuario: ObjectId('69d28fffae6a3bb9bbf987e0'),
  valoracion: 5,
  fecha: 1767260000
}
```

9. **Obtener los comentarios de una receta específica, ordenados por fecha.**

Para este último caso tendremos que hacer un índice en los comentarios que contenga el ID de la receta y la fecha en orden descendente, para obtener los comentarios ordenados de más reciente a menos:

```
db.comentarios.createIndex({receta: 1, fecha: -1})
```

Al ejecutar la consulta:

```
db.comentarios.find(
      {receta: ObjectId('69d2a3feae6a3bb9bbf987e3')}
).sort({fecha: -1}).explain("executionStats")
```

para ver los pasos que realiza Mongo para buscar los comentarios más recientes de la receta “Pan con tomate”, podemos ver en la clave “indexName” que se utiliza el índice que se acaba de crear (ya que solo tengo un comentario para esa receta):



Tras la realización de todos los apartados del ejercicio, considero que MongoDB no es la base de datos adecuada para almacenar la información requerida. Si bien es cierto que los índices permiten una velocidad elevada a la hora de obtener los resultados de las consultas, las características de los datos a almacenar no hacen que este tipo de bases de datos sean adecuadas. El principal motivo es la relación existente entre los usuarios y las recetas, no solo por publicarlas, sino por comentarlas y valorarlas.

Si bien es cierto que las colecciones de “comentarios” y de “valoraciones” podrían haberse introducido como claves dentro de la colección “recetas”, de forma parecida a como se hace la relación de “usuario marca como favorito receta” (y viceversa, dicha relación también se podría haber implementado como una colección nueva), seguiría existiendo la referencia del usuario, pues, como explico más arriba, usar la referencia (ya sea de usuario como este caso o de receta) permite la implementación del cambio de nombre de usuario sin la necesidad de tener que buscar donde se encuentra dicho nombre de usuario. La operación de

referenciar no existe por defecto en Mongo como sí puede existir en una base de datos SQL con el concepto de clave foránea.

Esto sumado al hecho de que todas las instancias de una determinada colección tengan la misma estructura hace, a mi parecer, aún más evidente la ventaja de una base de datos relacional frente a Mongo.

Comentario final

La práctica, al igual que la anterior, me ha parecido perfecta para entender más en profundidad cómo funciona Mongo, especialmente los índices, pues con las diapositivas del temario no terminaba de verle la utilidad que tienen. Sin embargo, se me ha hecho más pesada que la anterior, puede quizás por la cantidad de información que hay que detallar en el ejercicio 3 y por los 14 casos diferentes del ejercicio 2.

El ejercicio 2 me ha parecido útil para entender las funciones find y aggregate, pero he echado en falta algún ejercicio que sea para modificar o eliminar instancias de las colecciones. Personalmente, le daría un poco más de variedad al ejercicio para que no se sintiese repetitivo.

Con respecto al ejercicio 3, creo que es muy buen ejercicio, pero bastante largo debido a tener que poner tantos ejemplos/resultados, aunque entiendo que son necesarios para demostrar que el alumno entiende los conceptos. Todo lo relacionado con los índices me ha costado entenderlo, básicamente cómo funcionaban y la utilidad a la hora de realizar las consultas. He estado apoyándome de diversas páginas y sacando información de diferentes herramientas de inteligencia artificial para que me dieran más contexto sobre casos en los que estos índices se utilizaban y cómo ayudaban.

A la hora de desarrollar mi solución en este ejercicio también estuve buscando información sobre la posibilidad de hacer referencias a otros campos, así como del operador $lookup en la agregación para ver cómo de factible era la utilización de los IDs en otras colecciones. Si bien es cierto que es más complejo su uso que directamente los nombres de usuario o los títulos, como comento en el final del ejercicio 3, el uso de IDs nos ofrece más facilidades a la hora de implementar otras funcionalidades que considero que tiene más peso a la hora de decidir qué almacenar.

Otros de los planteamientos que tuve fue si utilizar los tipos de datos relacionados con las fechas implementados en Mongo o usar el timestamp de UNIX. Acabé decantándome por el uso del timestamp por facilidad de normalización que da con respecto a los diferentes clientes que usen la BD, ya sea para leer o escribir datos. Utilizar uno de los tipos requeriría definir un formato de fecha aceptada (que

debería de incluir huso horario incluso), mientras que el uso del timestamp es bastante simple: es el mismo para todo el mundo al estar referenciado a UTC y permite obtener la fecha según el huso horario de cada cliente, sin necesidad de tener que realizar transformaciones de uno a otro. Otra razón es que en mi entorno laboral trabajamos con eventos temporales y siempre usamos el timestamp, por lo que estoy relativamente familiarizado con el uso de estos.

Finalizando el ejercicio consideré que la posibilidad de extraer la relación “usuario marca favorito receta” podría haberla extraído en una nueva colección para tener mayor modularidad y poder operar mejor, como por ejemplo sacar la lista de usuarios que han marcado cierta receta como favorita.

También creo que debería de haber poblado un poco más la base de datos para darle más variedad y que los resultados de las consultas no fueran tan escuetos, pero por falta de tiempo me he visto obligado a dejarla con tan pocos datos.